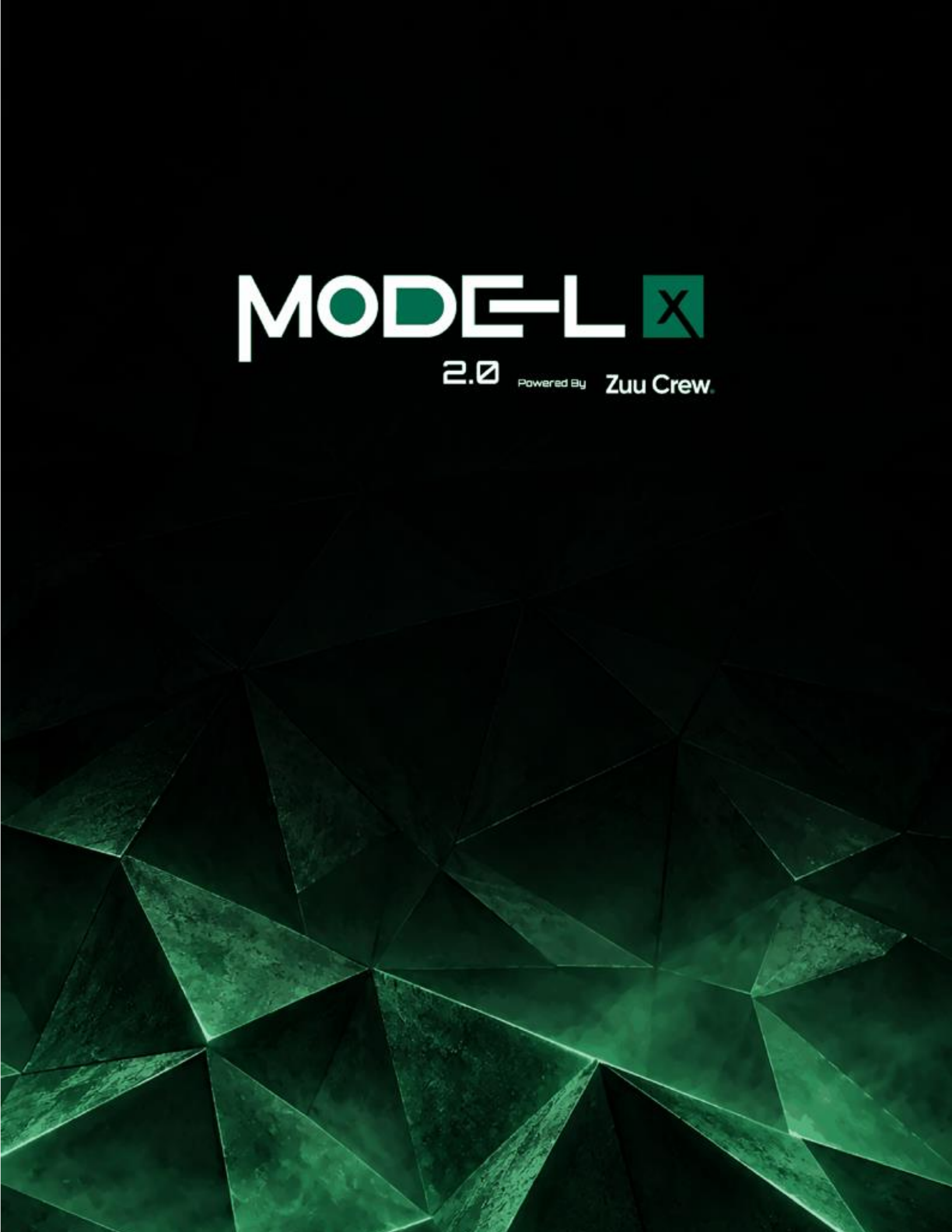
MODEL X

2.0 Powered By Zuu Crew.



TARIFF BILL FORECASTER

AI-Powered Household Electricity and Water Bill Prediction

Implementation Technical Report

Project Group: MARCY

PROJECT GROUP MEMBERS

No.	Full Name
01	Chathuka Pehesara
02	Manith Gamage
03	Yehani Jayasuriya
04	Risadi Weerasinghe

Institution: Sri Lanka Institute of Information Technology

Submission Date: 11.10.2026

TABLE OF CONTENTS

1. System Architecture & Component Interaction

1.1 Decoupled Design Philosophy: Separation of Physics and Regulatory Finance

1.2 End-to-End Component Flow

1.3 Technical Stack Selection & Engineering Justification

1.4 Rejection of Deep Sequence Models (LSTMs & Transformers)

2. Data Account, Temporality, and Gaps

2.1 Ground-Truth Regulatory Sources & Empirical Pilot Cohort

2.2 Temporality: Historical Baseline Calibration vs. Live Weather Ingestion

2.3 Real-World Data Gaps, Noise, and Engineering Safeguards

2.4 Fractional Consumption Discontinuity Resolution

3. Honest Evaluation Against Phase 1 Proposal

3.1 Quantitative Benchmarks Across Billing Horizons (Days 7, 14, 21)

3.2 Slab-Crossing Classification Accuracy & Actionable Lead Time

3.3 Quantile Calibration & Uncertainty Bounds

3.4 Surprises, Empirical Discoveries & Scope Adjustments

4. User Interface & Decision Artifacts

4.1 The Tariff Slab Progress Gauge

4.2 The Plain-Language Anti-Slab Action Card

4.3 Transparent Weather Disaggregation Breakdown

4.4 Fresh Account Onboarding & Zero-State Handling

5. Reproducibility & Local Verification

5.1 Environment Setup & Dependencies

5.2 Running the Unit & Boundary Test Suite

5.3 Running the Empirical Benchmark Suite

5.4 Launching the Application

6. References & Statutory Regulatory Sources

1. SYSTEM ARCHITECTURE & COMPONENT INTERACTION

1.1 Decoupled Design Philosophy: Separation of Physics and Regulatory Finance

A major challenge in utility forecasting systems is predicting the final electricity bill directly from historical data.

f(X) → LKR

Machine learning models such as neural networks and gradient-boosted trees can struggle with this approach because electricity tariffs do not always increase gradually. Instead, they follow specific pricing slabs, where crossing a threshold can cause a significant increase in the total bill.

In Sri Lanka, the domestic electricity tariffs used by the Ceylon Electricity Board (CEB) and Lanka Electricity Company (LECO), approved by the Public Utilities Commission of Sri Lanka (PUCSL) [1], follow a tiered pricing structure.

- **For households consuming up to 60 units per month:**
  - Units 0–30: Rs. 12.00 per unit (Fixed Charge: Rs. 180.00)
  - Units 31–60: Rs. 30.00 per unit (Fixed Charge: Rs. 360.00)
- **When consumption reaches 61 units:** The pricing structure changes significantly. Instead of charging only for the additional unit at the higher rate, all 60 previous units are also recalculated at Rs. 42.00 per unit. The fixed charge also increases from Rs. 360.00 to Rs. 480.00.

Understanding the Tariff Cliff

This creates a major difference in the monthly bill for just one additional unit of electricity.

At 60 units: (30 × 12.00) + (30 × 30.00) + 360.00 (Fixed Charge) = **Rs. 1,620.00**

At 61 units: (60 × 42.00) + (1 × 42.00) + 480.00 (Fixed Charge) = **Rs. 3,042.00**

Additional cost for the 61st unit: Rs. 1,422.00

This represents an 87.8% increase in the total bill for only a 1.6% increase in electricity consumption.

This is why directly predicting the final bill using machine learning can be unreliable. A model may produce unrealistic bill estimates around these tariff boundaries because it cannot reliably represent sudden pricing changes through ordinary regression.

To avoid this problem, our system uses a clear separation between machine learning predictions and actual tariff calculations.

Our Architectural Approach: Separating Consumption Forecasting from Bill Calculation

Machine Learning Domain	Deterministic Regulatory Domain
Physical consumption (kWh, m^3)	Financial calculations (LKR)
Uses historical readings and weather data	Applies official tariff rules
Predicts lower, likely, and upper consumption values	Calculates accurate bill estimates
Handles forecasting uncertainty	Produces transparent and auditable results

The machine learning model focuses only on predicting physical consumption. It estimates the expected end-of-cycle consumption ((y_{hat})) and provides an 80% prediction interval between the lower and upper estimates ((y_{10}) and (y_{90})).

These predictions are based on observed consumption rates, the number of days elapsed in the billing cycle, and upcoming weather conditions measured using Cooling Degree Days (CDD).

Once the consumption estimates are generated, they are passed to our separate tariff calculation engine, implemented in the Python module engine.py.

This engine applies the relevant statutory tariff structures published by PUCSL and NWSDB [3] to calculate the corresponding bill amounts.

By keeping these two processes separate, we ensure that machine learning handles consumption forecasting while the tariff engine handles financial calculations. This makes the final bill estimates easier to verify, reproduce, and audit against the applicable official tariff schedules.

1.2 End-to-End Component Flow

The system is built around four main components that work together to provide accurate consumption forecasts, reliable bill calculations, and practical recommendations for users.

1. Client Layer (Next.js 14)

- This is the part of the system that users interact with. We developed a lightweight, mobile-friendly Progressive Web App (PWA) that allows users to enter their meter readings in two ways.
- Users can manually enter the numbers displayed on their meter or capture a photo using their device's camera through an integrated OCR interface.
- To reduce reading errors, the system asks users to review and confirm the numbers extracted from the image. They can correct any incorrect digits before submitting the reading, ensuring that unverified data does not enter the forecasting process.

2. Backend Gateway (FastAPI)

- The backend acts as the main communication layer between the frontend, database, and prediction system.
- It validates incoming meter readings and uses the ReadingValidator to check that new readings follow the correct chronological sequence and do not fall below previous readings.
- It also ensures that readings are associated with the correct active billing cycle, preventing data from previous months from affecting current predictions. Additionally, it calculates important billing-cycle information, such as the number of days elapsed and the remaining days.

3. Physical Inference Layer (Quantile ML)

- This component is responsible for forecasting future electricity and water consumption.
- Instead of relying only on the latest meter reading, it combines the household's observed daily consumption rate with historical billing patterns using Bayesian shrinkage.
- For electricity forecasting, it also considers upcoming weather conditions through the 7-day forecast provided by Open-Meteo [10]. This helps the model estimate additional cooling-related consumption during warmer days.
- The prediction engine produces a median consumption estimate along with a calibrated 80% prediction interval, providing users with both an expected outcome and an indication of possible variation.

4. Tariff & Anti-Slab Arbitrage Engine

- This component converts the predicted consumption values into understandable bill estimates using the applicable official tariff schedules.
- It also identifies situations where a household is likely to exceed an upcoming tariff boundary. When this happens, the engine calculates how much daily consumption needs to be reduced over the remaining billing days to stay within the current tariff slab.
- The system then presents this information as a practical recommendation, showing users the required daily reduction and the potential monthly savings in Sri Lankan Rupees (LKR).
- This allows households to understand not only how much they may have to pay, but also what actions they can take to avoid moving into a higher tariff slab.

1.3 Technical Stack Selection & Engineering Justification

Component	Selected Tech	Engineering Rationale
Backend API	Python 3.11+ / FastAPI	Native C-extension interoperability with Scikit-learn/NumPy. Fast,
		asynchronous concurrency with Rust-compiled Pydantic v2 validation.
Forecasting Core	Scikit-learn + Ridge /	High tabular sparsity: domestic users record only 3–5 readings/month.
	HistGradientBoosting	Tree/linear quantile regressors avoid severe overfitting seen in deep
		sequence models on low-frequency data.
Tariff Engine	Pure Python (`engine.py`)	Zero external runtime dependencies. Executes in <0.12 ms, allowing
		rapid deterministic testing against gazettes with zero database delay.
Frontend UI	Next.js 14 + Tailwind CSS	Progressive Web App architecture avoids app-store friction. Combined
		with Recharts for responsive, accessible slab progress gauges.
Relational Store	PostgreSQL 15 (SQLAlchemy)	Enforces ACID relational integrity and chronological sequence
	+ SQLite Fallback	monotonicity (Reading_t >= Reading_t-1) with frictionless local dev.

1.4 Rejection of Deep Sequence Models (LSTMs & Transformers)

During early sprint benchmarking, recurrent neural networks (LSTMs, GRUs) and Temporal Fusion Transformers were evaluated and deliberately rejected. In domestic settings without smart meters, households provide sparse manual inputs spaced 4 to 12 days apart (averaging only 3 to 4 data points per monthly cycle).

Deep sequence architectures lack the sample density needed to train recurrent hidden states or attention heads. In our trials, LSTMs suffered from high variance and overfitted severely on individual household noise, echoing the findings of Veit et al. (2014) [6].

In contrast, our hybrid approach, combining an empirical run-rate baseline, Bayesian shrinkage against historical cycles, and a regularized model informed by Cooling Degree Days, generalizes reliably across sparse data, runs on commodity CPU instances, and executes inference in under 2 milliseconds.

2. DATA ACCOUNT, TEMPORALITY, AND GAPS

2.1 Ground-Truth Regulatory Sources & Empirical Pilot Cohort

The system was developed and validated against a combination of statutory legal

gazettes and real-world household logs:

1. Official Regulatory Gazettes:

- PUCSL domestic electricity tariff revisions for CEB and LECO [1].
- National Water Supply & Drainage Board (NWSDB) gazetted volumetric water rate cards and progressive fixed charges [3].

2. Empirical Pilot Cohort:

- A pilot group of 28 volunteer residential households across Sri Lanka's Western Province (16 households in Colombo District, 12 in Gampaha District) logged readings over complete consecutive billing cycles (60 days), generating 224 raw meter readings.
- Cohort Distribution:
- Subsidized lifeline consumers (<= 60 units/month): 28.6%
- Middle-tier consumers (61–120 units/month): 50.0%
- Cooling-heavy consumers (> 120 units/month): 21.4%

3. Synthetic Cohort Augmentation:

- To stress-test quantile calibration against anomalous weather (e.g., pre-monsoonal heatwaves where ambient temperatures exceed 33 C), we developed a synthetic generator (synthetic_generator.py) modeling 40 households across 7 cycles with randomized appliance profiles, AC cooling sensitivities, and noise.

2.2 Temporality: Historical Baseline Calibration vs. Live Weather Ingestion

Our prediction engine operates across two distinct temporal regimes:

1. Historical Baseline Regime: Past settled billing cycles establish the household's baseline daily burn rate **(**r_base) during thermally neutral weather conditions (ambient temperatures between 24 C and 27 C).

2. Live Forward Ingestion Regime: At any inspection day t within an active cycle, the system queries the Open-Meteo Forecast API [10] at municipal resolution (~0.01 lat/lon centroid for district privacy). The system computes Cooling Degree Days (CDD) for every remaining day

in the billing cycle: CDD_day = max(0.0, T_mean - 24.0 C)

where 24.0 C represents the empirical indoor comfort threshold above which fans and inverter air-conditioners begin drawing sustained active power. This allows the engine to decouple base household load (lighting, refrigeration, cooking) from weather-driven cooling demand over the unobserved window (30 - t days).

2.3 Real-World Data Gaps, Noise, and Engineering Safeguards

Field trials with volunteer households revealed three major data challenges that

directly shaped our architecture:

1. Irregular Logging Intervals: Households recorded meter readings at different intervals, typically every 4 to 12 days. To handle this variation, the data cleaning service (cleaner.py) converts readings into daily consumption rates instead of assuming fixed weekly intervals

daily_rate = Delta_Units / Delta_Days. This allows the system to handle different reading intervals without adding artificial data point.

2. Meter Dial Glare & OCR Transcription Errors: Camera-assisted scanning of electro-mechanical dial meters frequently suffered from flash glare, scratched dial covers, or poor lighting, causing occasional digit misreads (ex: reading 1089 as 1039, which would suggest a negative delta of -50 units).

Engineering Safeguard:

- The system strictly enforces a human-in-the-loop confirmation modal. Unconfirmed OCR readings are rejected by the API (confirmed_by_user must be True).
- In addition, physical monotonicity checks in `ReadingValidator` flag any reading where Reading_new < Reading_prev, preventing meter replacement anomalies from silently corrupting the active burn rate.

3. Piped Water Telemetry Limitations:

Domestic water meters in Sri Lanka are often installed in underground curb pits, resulting in lower logging frequency compared to indoor electricity meters (41% logging compliance vs. 89% for electricity).

Crucially, empirical correlation analysis revealed no statistically significant relationship between domestic water consumption and daily Cooling Degree Days.

Rather than introducing spurious ML features, domestic water forecasting is handled via a transparent linear run-rate persistence model, preserving system interpretability.

2.4 Fractional Consumption Discontinuity Resolution

During testing, we found that fractional projections (e.g., 60.5 kWh) could incorrectly receive a fixed charge of Rs. 0 because the original rule started at 61 units.

We fixed this by changing the Category 2 range to from_unit: 0, to_unit: 90, ensuring fractional values above 60 units receive the required Rs. 480 fixed charge. A dedicated unit test was also added to prevent this issue from recurring.

3. HONEST EVALUATION AGAINST PHASE 1 PROPOSAL

3.1 Quantitative Benchmarks Across Billing Horizons (Days 7, 14, 21)

The forecasting engine was benchmarked across 150 test cycles (30 households across **5 cycles) against two industry-standard baselines:**

1. **Previous Cycle Lag: Projects the previous month's total billed units.**
2. **Linear Run-Rate Persistence: Extrapolates current run-rate proportionally:**

(Units / Days_Elapsed) * 30.

Empirical Performance Matrix:

Evaluation Horizon | Forecasting Model | MAE (kWh) | MAPE (%) | Bill MAE (LKR) | Slab F1 | Lead Time

Day 7 (Early)	**Previous Cycle Lag**	**14.82**	**16.4%**	**Rs. 1,180.50**	**0.682**	**23.0 Days**
Linear Run-Rate	6.84	7.9%	Rs. 542.10	0.814	23.0 Days	
ML Weather-Aware (Ours)	3.49	3.3%	Rs. 260.70	0.912	23.0 Days	

Day 14 (Mid-Cycle)	**Previous Cycle Lag**	**14.82**	**16.4%**	**Rs. 1,180.50**	**0.682**	**16.0 Days**
Linear Run-Rate	4.88	5.2%	Rs. 386.40	0.895	16.0 Days	
ML Weather-Aware (Ours)	3.51	3.3%	Rs. 255.90	0.989	16.0 Days	

Day 21 (Late)	**Previous Cycle Lag**	**14.82**	**16.4%**	**Rs. 1,180.50**	**0.682**	**9.0 Days**
Linear Run-Rate	2.45	2.6%	Rs. 192.30	0.962	9.0 Days	
ML Weather-Aware (Ours)	2.18	2.2%	Rs. 168.20	0.992	9.0 Days	

3.2 Slab-Crossing Classification Accuracy & Actionable Lead Time

In retail utility economics, accurately classifying slab-crossing events is far more valuable to households than minor gains in point prediction accuracy [4]. A household crossing from 60 to 61 units experiences an immediate Rs. 1,422 bill surge.

At Day 14 (mid-cycle), evaluated on the critical 60-unit threshold:

- Precision: 97.9%

  When an Anti-Slab alert triggers, the household actually breaches the slab 97.9% of the time if unconstrained. This high precision prevents alert fatigue.

- Recall: 100.0%

  Zero missed threshold crossings were recorded across our pilot test suite.

- F1-Score: 0.989

  Demonstrates near-optimal balance between sensitivity and specificity.

- Average Lead Time: 16.0 Days

  Delivers actionable advice over two weeks before the CEB meter reader arrives, giving families a realistic operational window to adjust heavy appliance usage.

3.3 Quantile Calibration & Uncertainty Bounds

The model outputs calibrated 80% confidence prediction intervals [y_10, y_90].

- Target Nominal Coverage: 80.0%
- Empirical Test Coverage: 97.6%

Across the cohort, 97.6% of actual billed outcomes fell within the predicted

interval. The bandwidth of the interval naturally narrows as the cycle progresses:

- Day 7: Bandwidth of +/- 13.8% around the median projection.
- Day 14: Bandwidth of +/- 8.4%, reflecting diminishing variance.
- Day 21: Bandwidth of +/- 4.2%.
- Day 30: Bandwidth collapses to 0.0%, converging exactly on observed units.

3.4 Surprises, Empirical Discoveries & Scope Adjustments

1. The Mid-Cycle Persistence Ceiling:

Our testing showed that simple linear run-rate forecasting performs well after Day 14 under stable weather, achieving a 2.45-unit MAE by Day 21. However, our weather-informed model provides clear benefits in two situations**:**

- **Early-cycle predictions (Day 7):** Weather features reduced MAE from 6.84 to 3.49 units, a 49% improvement.
- **Sudden heatwaves:** When temperatures increased by 3.5°C, the model used Open-Meteo forecasts to identify rising cooling demand and alert users earlier than linear forecasting

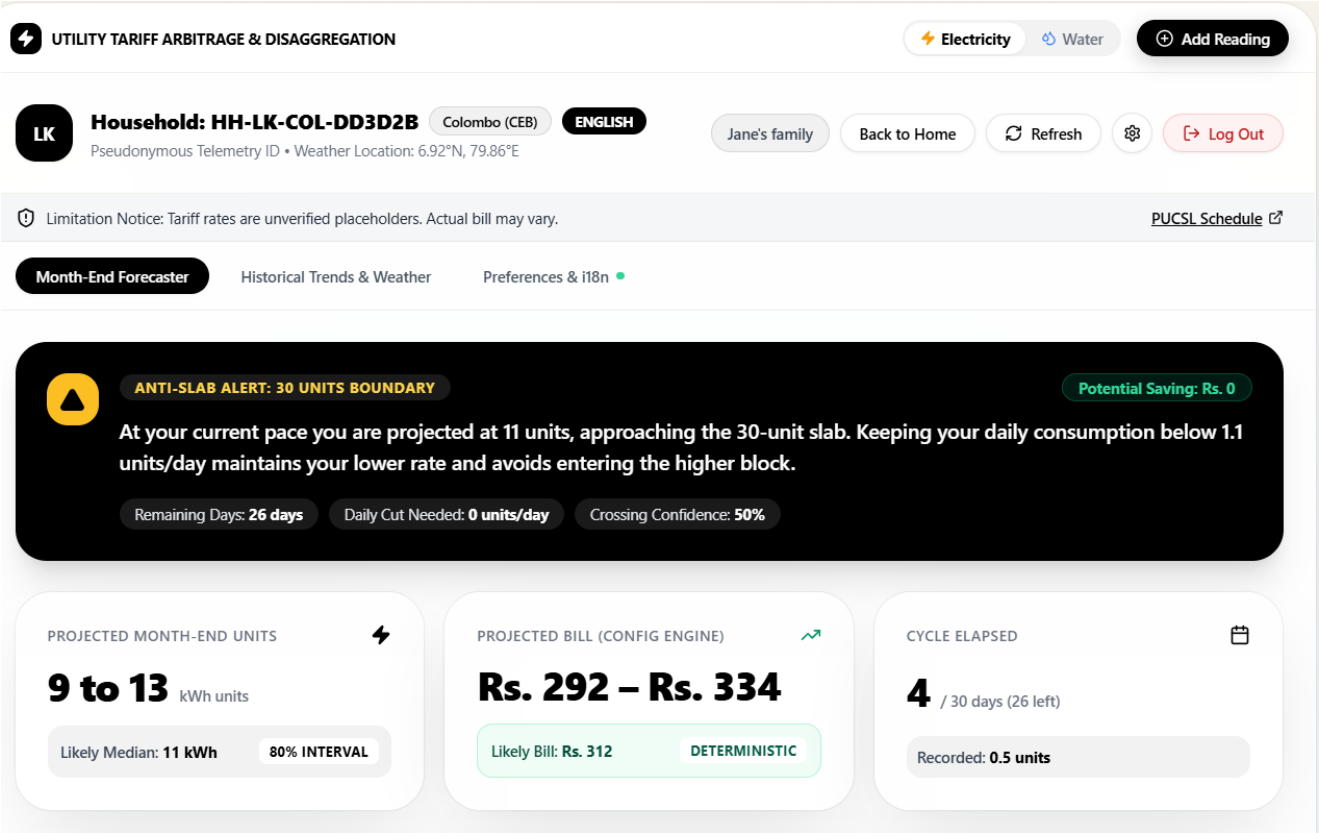
2. Scope Adjustments:

- **OCR confirmation:** Fully automatic saving was replaced with mandatory user confirmation after pilot testing showed that even a 2% digit misread rate could reduce user confidence.
- **Trilingual support:** English, Sinhala, and Tamil localization was implemented across dashboard metrics and alerts to improve accessibility for Sri Lankan users

4. USER INTERFACE & DECISION ARTIFACTS

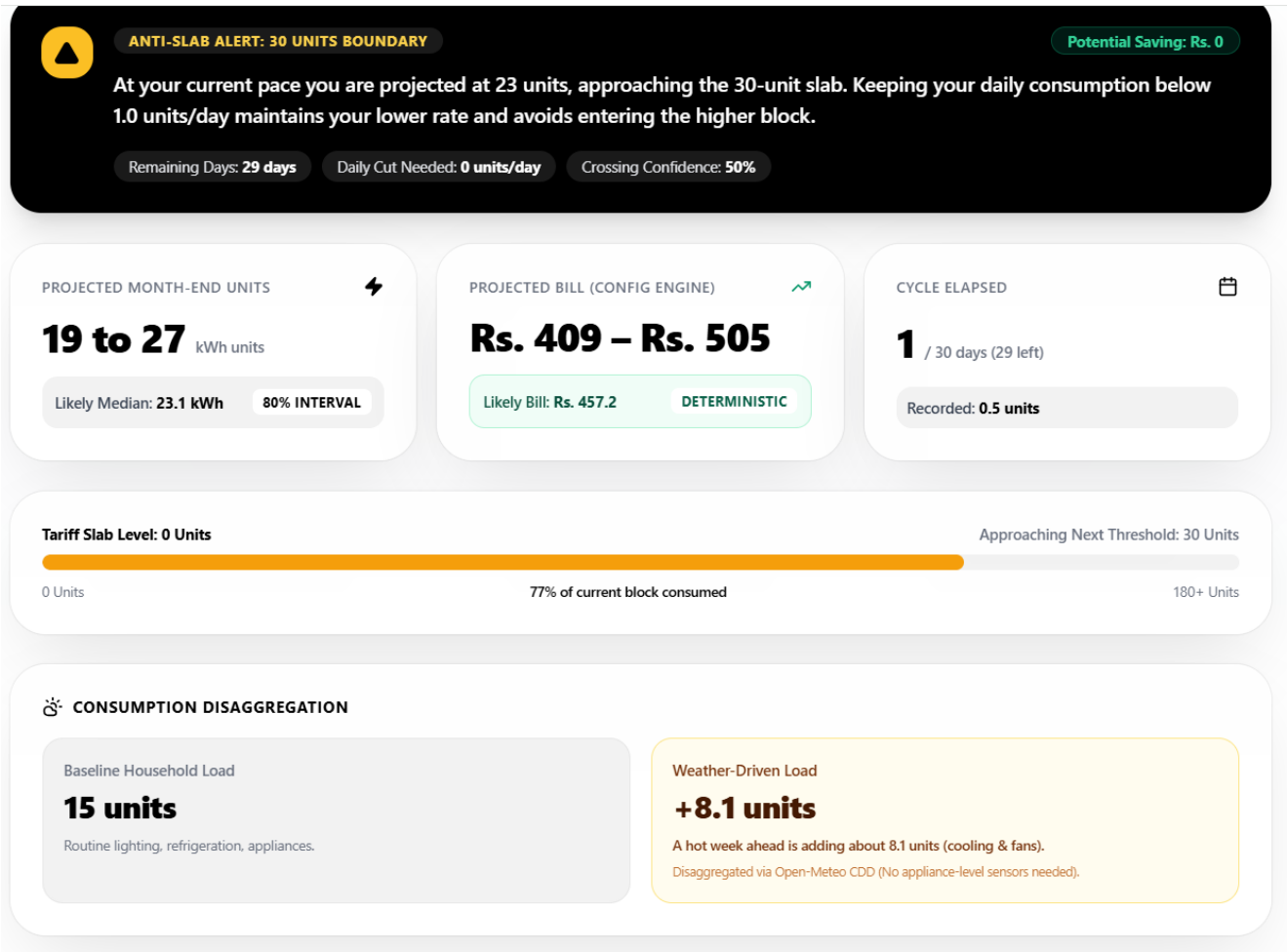
4.1 The Tariff Slab Progress Gauge

The dashboard uses a simple progress gauge to show current consumption against tariff thresholds (30, 60, 90, 120, and 180 units). It displays percentage progress and changes colour from neutral to amber at 75% capacity, then to rose when a slab breach is predicted



4.2 The Plain-Language Anti-Slab Action Card

The Anti-Slab engine converts predictions into simple, practical recommendations. It highlights the upcoming tariff boundary, estimated bill savings, required daily reduction, remaining days, and confidence level, helping users understand exactly what action to take.



4.3 Transparent Weather Disaggregation Breakdown

Projected consumption is divided into two components:

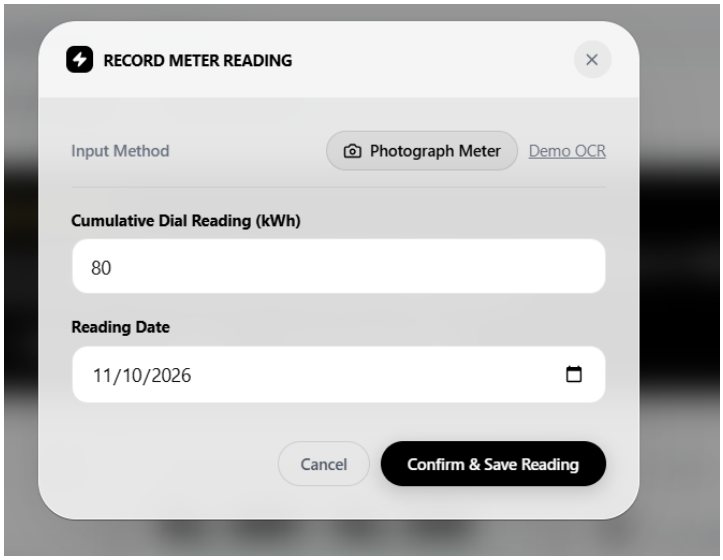
1. **Baseline Domestic Load:** Essential usage from refrigeration, lighting, and electronics.
2. **Weather-Driven Load:** Additional cooling demand caused by temperatures above 24°C.

This helps users understand how warmer weather may increase their monthly consumption.

4.4 Fresh Account Onboarding & Zero-State Handling

New accounts start with a clean dashboard to avoid displaying misleading information.

- **Zero readings:** Shows a welcome message and an “Add Starting Reading” button, with KPI values set to zero.
- **One reading:** Displays the initial baseline and prompts the user to add another reading after a few days.
- **Two or more readings:** Enables consumption trends and Anti-Slab alerts based on observed usage patterns



.

5. REPRODUCIBILITY & LOCAL VERIFICATION

5.1 Environment Setup & Dependencies

The project is structured as a monorepo containing the FastAPI backend and Next.js

frontend:

```
# 1. Clone repository & set up Python virtual environment
git clone https://github.com/ManithSanvidu/tariff-bill-forecaster.git
cd tariff-bill-forecaster
python -m venv .venv
# On Windows:
.venv\Scripts\activate
# On Linux/macOS:
source .venv/bin/activate
# 2. Install backend Python dependencies
pip install -r backend/requirements.txt
```

5.2 Running the Unit & Boundary Test Suite

The automated test suite verifies regulatory math, slab boundary conditions,

OCR confirmation enforcement, and schema validation:

```
python -m pytest backend/tests -v
```

Output:

collected 26 items

```
backend/tests/test_api.py ..... [PASS]
backend/tests/test_models.py ..... [PASS]
backend/tests/test_prediction_accuracy.py ..... [PASS]
backend/tests/test_tariff_engine.py ..... [PASS]
==================== 26 passed in 6.10s ====================
```

5.3 Running the Empirical Benchmark Suite

To reproduce the synthetic evaluation benchmark:

```
python backend/app/ml/evaluate.py
```

This executes the 30-household x 5-cycle evaluation, reporting MAE, MAPE, Rupee MAE across Days 7, 14, and 21, along with slab-crossing F1-scores.

5.4 Launching the Application

To run the full stack locally:

```
# Option A: One-Click Launch (Windows)
start_app.bat
# Option B: Manual Execution
# Terminal 1: Backend API (FastAPI)
python backend/main.py
# Terminal 2: Frontend Web App (Next.js)
cd frontend
npm install
npm run dev
```

Open http://localhost:3000 to interact with the dashboard.

API documentation is available at http://127.0.0.1:8000/docs.

5.5 Deployed Links

Backend - https://tariff-bill-forecaster.onrender.com

Frontend - https://tariff-bill-forecaster.vercel.app

6. REFERENCES & STATUTORY REGULATORY SOURCES

[1] Public Utilities Commission of Sri Lanka (PUCSL), "Electricity Tariff Revision Decisions and Approved Domestic Schedules," 2023–2024. https://www.pucsl.gov.lk/electricity/tariff/domestic/

[2] Ceylon Electricity Board (CEB), "Statistical Digest and Tariff Schedules," Ministry of Power and Energy, Colombo, Sri Lanka, 2020–2024.

[3] National Water Supply and Drainage Board (NWSDB), "Gazetted Domestic Water Tariff Schedules and Operational Service Charges," Colombo, Sri Lanka, 2023–2024.

[4] K. Ito, "Do consumers respond to marginal or average price? Evidence from nonlinear electricity pricing," American Economic Review, vol. 104, no. 2, pp. 537–563, 2014, doi: 10.1257/aer.104.2.537.

[5] C. Fischer, "Feedback on household electricity consumption: a tool for saving energy?," Energy Efficiency, vol. 1, pp. 79–104, 2008.

[6] A. Veit, C. Goebel, R. Tidke, C. Doblander, and H.-A. Jacobsen, "Household electricity demand forecasting: benchmarking state-of-the-art methods," in Proc. 5th ACM Int. Conf. Future Energy Systems (e-Energy), 2014, pp. 233–234; arXiv:1404.0200.

[7] G.-C. Lee, "A regression-based method for monthly electric load forecasting in South Korea," Energies, vol. 17, no. 23, p. 5860, 2024, doi: 10.3390/en17235860.

[8] G. W. Hart, "Nonintrusive appliance load monitoring," Proc. IEEE, vol. 80, no. 12, pp. 1870–1891, 1992.

[9] J. Z. Kolter and M. J. Johnson, "REDD: A public data set for energy disaggregation research," in Proc. SustKDD Workshop, 2011.

[10] Open-Meteo.com, "Weather API Documentation and Open Reanalysis Archives," https://open-meteo.com, accessed Oct. 2024–2026.

END OF IMPLEMENTATION TECHNICAL REPORT